starts_with and ends_with

Document #:	P1659R2
Date:	2021-02-10
Project:	Programming Language C++
Audience:	Library Wording

Reply-to:	Christopher Di Bella <cjdb.ns@gmail.com>

Contents

§ 1 Abstract

This proposal seeks to add `std::ranges::starts_with` and `std::ranges::ends_with`, which would work on arbitrary ranges, and also answer questions such as “are the starting elements of `r1` *less than* the elements of `r2`?” and “are the final elements of `r1` *greater than* the elements of `r2`?”. It also proposes a change to SD-8.

§ 2 Change log

§ 2.1 R1 → R2

— Replaces *Effects* with *Returns*.

2.2 R0 → R1

— New document layout.

— Adds feature test macro.

— Replaces `PascalCase` concept names with `snake_case` concept names.

— Renames `Comp` to `Pred`.

— Removes paragraph 2 from [alg.starts_with].

3 Motivation

C++20 introduces `basic_string_view::starts_with`, `basic_string_view::ends_with`, `basic_string::starts_with`, and `basic_string::ends_with`. Both `basic_string` and `basic_string_view` are perculiar container-like types, with many member functions that duplicate algorithm functionality with minor interface changes (e.g. `compare`, `copy`, `find`, `rfind`, `find_first_of`, etc.). [P0457R2] §5.1 notes that the decision to add `starts_with` and `ends_with` as member functions was because (a) it is consistent with the aforementioned member functions, and (b) that P0457 agrees with [N3609] in that `starts_with(haystack, needle)` is ambiguous with `starts_with(needle, haystack)`. It should be noted that neither N3609, nor P0457 identified whether or not they were talking about non-member functions that only operate on string types, or if they were discussing an algorithm, but the LEWG minutes for P0457 reveal some LWEG interest in making them algorithms.

Although there is prior art with respect to `basic_string`'s member functions, the author expresses concern that our string types have a large set of member functions that either duplicate the algorithms or are directly incompatible with them, and thus limit the amount of composition that's possible. Templates are one of C++'s greatest strengths, and with iterators, we have an extremely extensible and powerful generic programming model. We should take advantage of this model wherever possible to ensure that we do not paint ourselves into a corner with a single type.

At present, it isn't *immediately* possible to query whether or not any range – other than a standard string type – is prefixed or suffixed by another range. To do so, one must use `mismatch` or `equal`, and at least in the case of `ends_with`, `ranges::next` (C++20).

It is interesting to note that, since `starts_with` and `ends_with` can be implemented using `mismatch`, we are able to query more than just "are the first *n* elements of range one the same as the entirety of range two?": we're also able to query "are the first *n* elements of range one all greater than the entirety of range two?", where *n* is the distance of the second range. See §1.1 for examples. This is something that the string-based member functions are not able to do, although the author acknowledges that text processing may not require this functionality.

Concerns were outlined in both N3609 and P0457 about the ambiguity of whether we are performing `starts_with(haystack, needle)` or `starts_with(needle, haystack)`. There is prior art in the algorithms library that makes the first range the subject of the operation: `mismatch`, `equal`, `search`, `find_first_of`, and `lexicographical_compare` all take the form `algorithm(haystack, needle)`, so the author remains unconvinced about the ambiguity.

<table>
<tr><th>Before</th><th>After</th></tr>
<tr><td>

```
auto some_ints = view::iota(0, 50);
auto some_more_ints = view::iota(0,
        30);
if (ranges::mismatch(some_ints,
        some_more_ints).in2 ==
        end(some_more_ints)) {
   // do something
}
```

</td><td>

```
auto some_ints = view::iota(0, 50);
auto some_more_ints = view::iota(0,
        30);
if (ranges::starts_with(some_ints,
        some_more_ints)) {
   // do something
}
```

</td></tr>
<tr><td>

```
auto some_ints = view::iota(0, 50);
auto some_more_ints = view::iota(0,
        30);
{
   auto some_ints_tail = subrange{
     next(begin(some_ints),
        distance(some_more_ints),
        end(some_ints)),
     end(some_ints)
   };
   if (not equal(some_ints_tail,
        some_more_ints)) {
      // do something
   }
}
```

</td><td>

```
auto some_ints = view::iota(0, 50);
auto some_more_ints = view::iota(0,
        30);
if (not ranges::ends_with(some_ints,
        some_more_ints)) {
   // do something
}
```

</td></tr>
</table>

§ 3.1 Example usage

```
auto script = u8"OBI-WAN: Hello, there!\n"
              u8"GENERAL GRIEVOUS: General Kenobi, you are a bold one."sv;
namespace ranges = std::ranges;
ranges::starts_with(script, u8"OBI-WAN"sv);               // returns true
ranges::starts_with(script, u8"ABCDEFG"sv);               // returns false
ranges::starts_with(script, u8"ABCDEFG"sv, ranges::less); // returns true

namespace view = ranges::view;
ranges::ends_with(view::iota(0, 50), view::iota(30) | view::take(20));
        // returns true
ranges::ends_with(view::iota(0, 50), view::iota(30) | view::take(50));
        // returns false
ranges::ends_with(view::iota(0, 50), view::iota(-50, -40),
        ranges::greater); // returns true
```

§ 4 Proposed changes to SD-8

In response to the concerns outlined in the motivation section of this document, the author would like to request that LEWG consider discussing adopting in SD-8, a policy of ensuring that options for algorithms are preferred when a proposal to add a member function to a container-like type is considered.

§ 5 Proposed changes to C++23

Add the following to 17.3.2 [version.syn]:

```
#define __cpp_lib_ranges_starts_ends_with 202007L // also in <algorithm>
```

Add the following text to 25.4 [algorithm.syn]:

```
...
template<class forward_iterator, class Searcher>
  constexpr forward_iterator
    search(forward_iterator first, forward_iterator last, const Searcher&
        searcher);

+ // [alg.starts_with], starts_with
+ template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
+          class Pred = ranges::equal_to, class Proj1 = identity, class
        Proj2 = identity>
+   requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
+ constexpr bool ranges::starts_with(I1 first1, S1 last1, I2 first2, S2
        last2, Pred pred = {},
+                                    Proj1 proj1 = {}, Proj2 proj2 = {});
+ template<input_range R1, input_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity,
+          class Proj2 = identity>
+   requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
+ constexpr bool ranges::starts_with(R1&& r1, R2&& r2, Pred pred = {},
+                                    Proj1 proj1 = {}, Proj2 proj2 = {});
+
+ // [alg.ends_with], ends_with
+ template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
+          class Pred = ranges::equal_to, class Proj1 = identity, class
        Proj2 = identity>
+   requires (forward_iterator<I1> || sized_sentinel_for<S1, I1>) &&
+            (forward_iterator<I2> || sized_sentinel_for<S2, I2>) &&
+            indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
+ constexpr bool ranges::ends_with(I1 first1, S1 last1, I2 first2, S2
        last2, Pred pred = {},
+                                  Proj1 proj1 = {}, Proj2 proj2 = {});
+ template<input_range R1, input_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity,
+          class Proj2 = identity>
+   requires (forward_range<R1> || sized_range<R1>) &&
```

```
+            (forward_range<R2> || sized_range<R2>) &&
+            indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
+ constexpr bool ranges::ends_with(R1&& r1, R2&& r2, Pred pred = {},
+                                  Proj1 proj1 = {}, Proj2 proj2 = {});

// [alg.modifying.operations], mutating sequence operations
// [alg.copy], copy
...
```

Add the following two sections to 25.6 [alg.nonmodifying]:

§ 5.1 25.6.14 Starts with [alg.starts.with]

```
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class
        Proj2 = identity>
  requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ranges::starts_with(I1 first1, S1 last1, I2 first2, S2
        last2, Pred pred = {},
                                   Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity,
         class Proj2 = identity>
  requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
constexpr bool ranges::starts_with(R1&& r1, R2&& r2, Pred pred = {},
                                   Proj1 proj1 = {}, Proj2 proj2 = {});
```

1 *Returns*: `ranges::mismatch(first1, last1, first2, last2, pred, proj1, proj2).in2 == last2`

§ 5.2 25.6.15 Ends with [alg.ends.with]

```
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2,
        sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class
        Proj2 = identity>
  requires (forward_iterator<I1> || sized_sentinel_for<S1, I1>) &&
           (forward_iterator<I2> || sized_sentinel_for<S2, I2>) &&
           indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ranges::ends_with(I1 first1, S1 last1, I2 first2, S2 last2,
        Pred pred = {},
                                 Proj1 proj1 = {}, Proj2 proj2 = {});
```

1 Let `N1` be `last1 - first1` and `N2` be `last2 - first2`.

2 *Returns*: `false` if `N1 < N2`, otherwise `ranges::equal(first1 + (N1 - N2), last1, first2, last2, pred, proj1, proj2)`.

```
template<input_range R1, input_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity,
         class Proj2 = identity>
  requires (forward_range<R1> || sized_range<R1>) &&
```

```
          (forward_range<R2> || sized_range<R2>) &&
          indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
constexpr bool ranges::ends_with(R1&& r1, R2&& r2, Pred pred = {},
                                 Proj1 proj1 = {}, Proj2 proj2 = {});
```

3 Let `N1` be `ranges::distance(r1)` and `N2` be `ranges::distance(r2)`.

4 *Returns*: `false` if `N1 < N2`, otherwise `ranges::equal(ranges::drop_view(r1, N1 - N2), r2, pred, proj1, proj2)`.

§ 6 Reference implementation

Both [starts_with] and [ends_with] have been implemented in range-v3.

§ 7 Acknowledgements

The author would like to thank Arien Judge for reviewing the proposal, Johel Ernesto Guerrero Peña for providing an implementation for `ends_with`, and Eric Niebler for merging the respective pull requests to range-v3.

§ 8 References

[ends_with] Johel Ernesto Guerrero Peña and Eric Niebler. ranges::ends_with. https://git.io/fjzq0

[N3609] Jeffrey Yasskin. 2013-03-15. string_view: a non-owning reference to a string, revision 3. https://wg21.link/n3609

[P0457R2] Mikhail Maltsev. 2017-11-11. String Prefix and Suffix Checking. https://wg21.link/p0457r2

[starts_with] Christopher Di Bella and Eric Niebler. ranges::starts_with. https://git.io/fjzqR